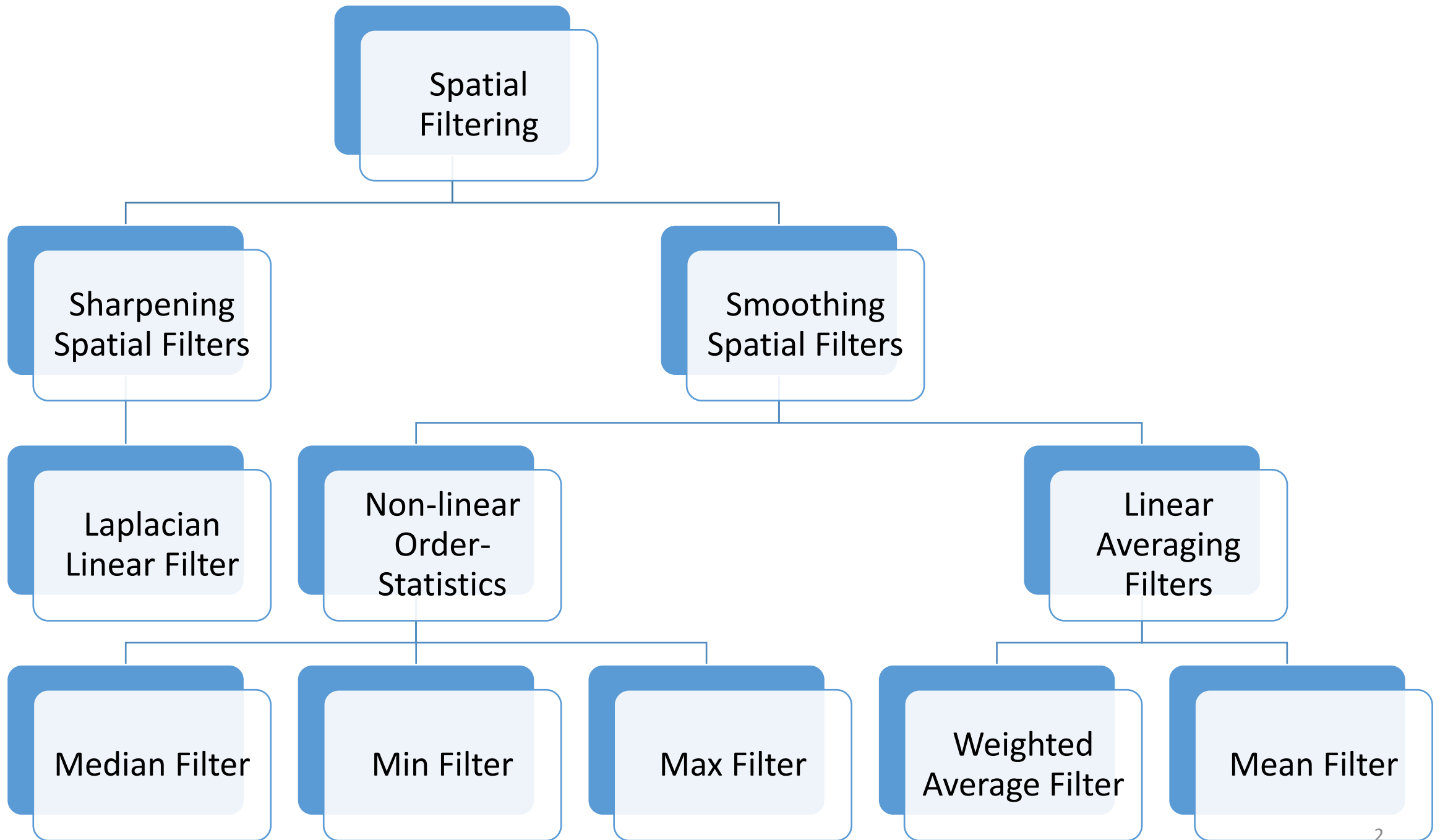


Shyamoli Textile Engineering College
Department of Computer Science and Engineering
CSE-4237: Digital Image Processing

Lec-6: Image Enhancement in the Spatial Domain

Digital Image Processing by Rafael C. Gonzalez and Richard Eugene Woods (Chapter-3)

Md. Ariful Islam
Assistant Professor
Dept. of Robotics and Mechatronics Engineering
University of Dhaka



7.3 Smoothing Spatial Filters

Smoothing filters are used for blurring and for noise reduction.

Why Blurring?

Blurring is used in preprocessing steps, such as removal of small details from an image prior to (large) object extraction, and bridging of small gaps in lines or curves.

How Noise Reduction?

Noise reduction can be accomplished by blurring with a linear filter and also by nonlinear filtering.

7.3.1 Smoothing Linear Filters (Mean Filter)

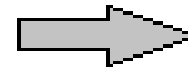
The output (response) of a smoothing, linear spatial filter is simply the average of the pixels contained in the neighborhood of the filter mask.

These filters sometimes are called averaging filters. They also are referred to a lowpass filters.

$$R = \frac{1}{9} \sum_{i=1}^9 z_i$$

unfiltered values

5	3	6
2	9	1
8	4	7



mean filtered

*	*	*
*	5	*
*	*	*

$$5 + 3 + 6 + 2 + 9 + 1 + 8 + 4 + 7 = 45$$

$$45 / 9 = 5$$

Example: Consider the following 5×5 image matrix:

$$\begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 60 & 70 & 80 & 90 & 100 \\ 110 & 120 & 130 & 140 & 150 \\ 160 & 170 & 180 & 190 & 200 \\ 210 & 220 & 230 & 240 & 250 \end{bmatrix}$$

Perform a mean filter using a simple 3×3 kernel on a 5×5 grayscale image.

Solution:

❑ The 3×3 window centered at (2,2) includes:

$$\begin{bmatrix} 10 & 20 & 30 \\ 60 & 70 & 80 \\ 110 & 120 & 130 \end{bmatrix}$$

Compute the sum of the kernel values:

$$10 + 20 + 30 + 60 + 70 + 80 + 110 + 120 + 130 = 630$$

Divide by the number of pixels ($3 \times 3 = 9$):

$$\text{Mean} = \frac{630}{9} = 70$$

The new pixel value for position (2,2) is 70.

□ The 3×3 window centered at (3,3) includes:

$$\begin{bmatrix} 70 & 80 & 90 \\ 120 & 130 & 140 \\ 170 & 180 & 190 \end{bmatrix}$$

Compute the sum:

$$70 + 80 + 90 + 120 + 130 + 140 + 170 + 180 + 190 = 1180$$

Divide by the number of pixels ($3 \times 3 = 9$):

$$\text{Mean} = \frac{1180}{9} = 131.1 \approx 131$$

The new pixel value for position (3,3) is 131.

After applying the mean filter to the entire image (ignoring edges), the result is:

$$\begin{bmatrix} \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & 70 & 80 & 90 & \cdot \\ \cdot & 120 & 131 & 140 & \cdot \\ \cdot & 170 & 180 & 190 & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix}$$

Original image, of size
500 X 500 pixels.



Results of smoothing
with square averaging
filter masks of sizes
 $n=3$



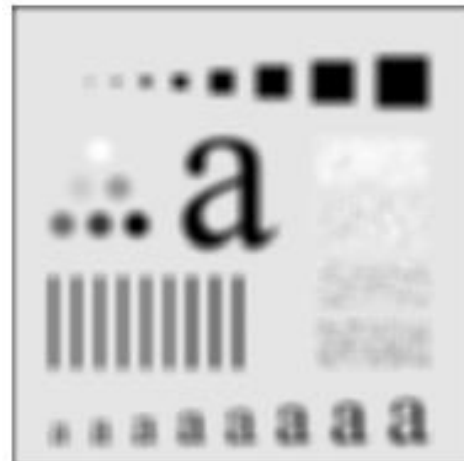
Results of smoothing
with square averaging
filter masks of sizes
 $n=5$



Results of smoothing
with square averaging
filter masks of sizes
 $n=9$



Results of smoothing
with square averaging
filter masks of sizes
 $n=15$



Results of smoothing
with square averaging
filter masks of sizes
 $n=35$



7.3.2 Smoothing Linear Filters (Weighted Average Filter)

The weighted average filter is a variation of the mean filter where each pixel in the kernel is assigned a weight, and the filtered value is computed as a weighted sum of the pixel values.

This filter is often used for smoothing while giving more importance to certain pixels, such as the central pixel, to preserve edges and details better than a regular mean filter.

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} \quad \frac{1}{16} \times \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 2 & 4 & 2 \\ \hline 1 & 2 & 1 \\ \hline \end{array}$$

How It Works

- ❖ Define a kernel (e.g., 3×3) with weights for each position.
- ❖ The sum of the weights is usually normalized to 1.
- ❖ For each pixel in the image, overlay the kernel on the pixel and its neighbors.
- ❖ Multiply each pixel value by the corresponding weight in the kernel.
- ❖ Sum the weighted values and assign this sum to the central pixel in the output image.

Example

Input Image:

We will use a 5×5 grayscale image:

$$\text{Image} = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 60 & 70 & 80 & 90 & 100 \\ 110 & 120 & 130 & 140 & 150 \\ 160 & 170 & 180 & 190 & 200 \\ 210 & 220 & 230 & 240 & 250 \end{bmatrix}$$

Kernel (Weighted 3×3):

A commonly used kernel for weighted averaging is:

$$\text{Kernel} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

The central pixel has the highest weight (4).

The weights sum to 16, ensuring normalization.

Calculation for Central Pixel (Pixel at Position (2,2)):

Kernel overlaid on the window centered at (2,2):

$$\text{Window} = \begin{bmatrix} 10 & 20 & 30 \\ 60 & 70 & 80 \\ 110 & 120 & 130 \end{bmatrix}$$

Compute the weighted sum:

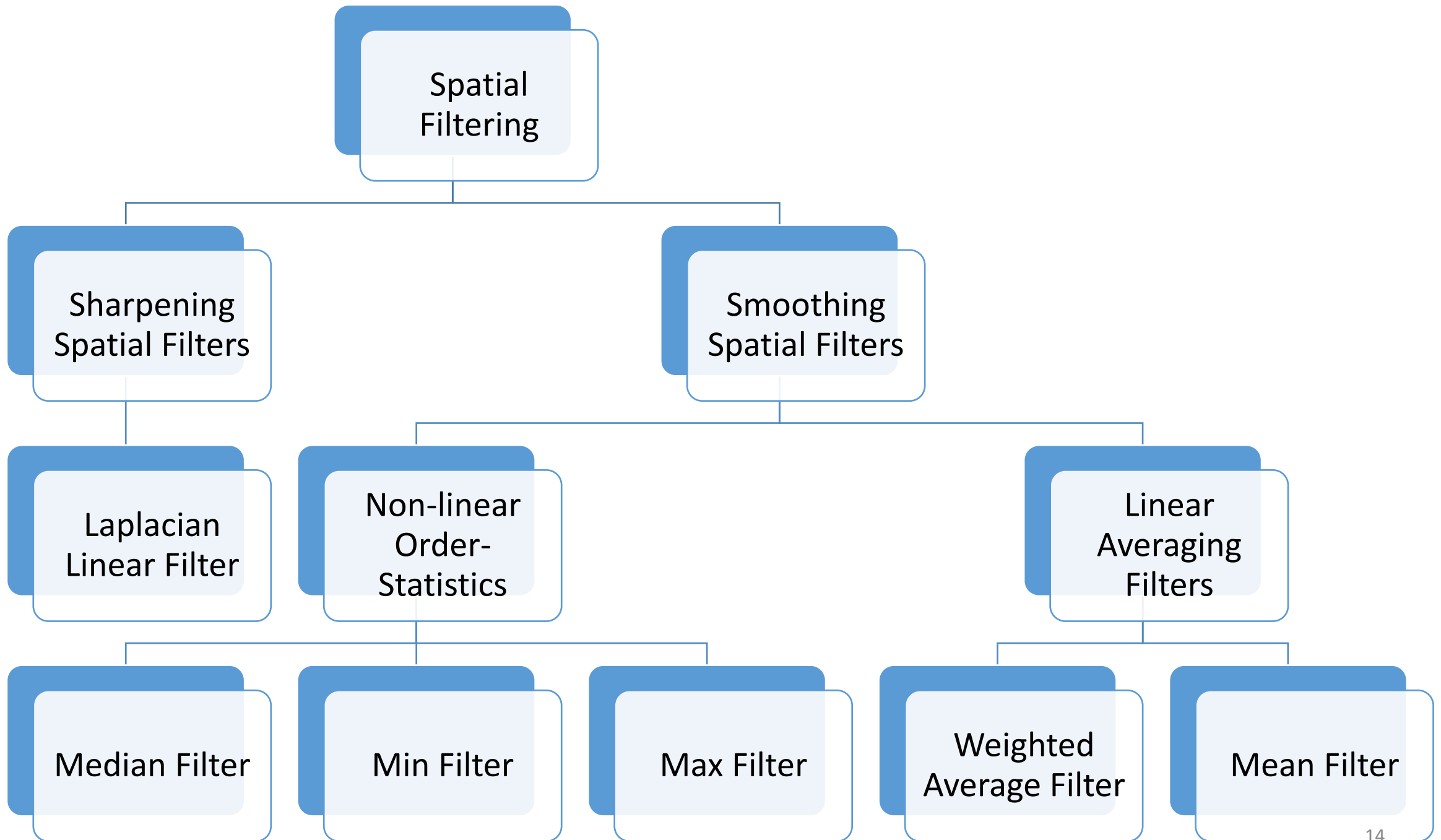
$$\text{Sum} = (10 \cdot 1) + (20 \cdot 2) + (30 \cdot 1) + (60 \cdot 2) + (70 \cdot 4) + (80 \cdot 2) + (110 \cdot 1) + (120 \cdot 2) + (130 \cdot 1)$$

$$\text{Sum} = 10 + 40 + 30 + 120 + 280 + 160 + 110 + 240 + 130 = 1120$$

Divide by the sum of weights (16):

$$\text{Weighted Average} = \frac{1120}{16} = 70$$

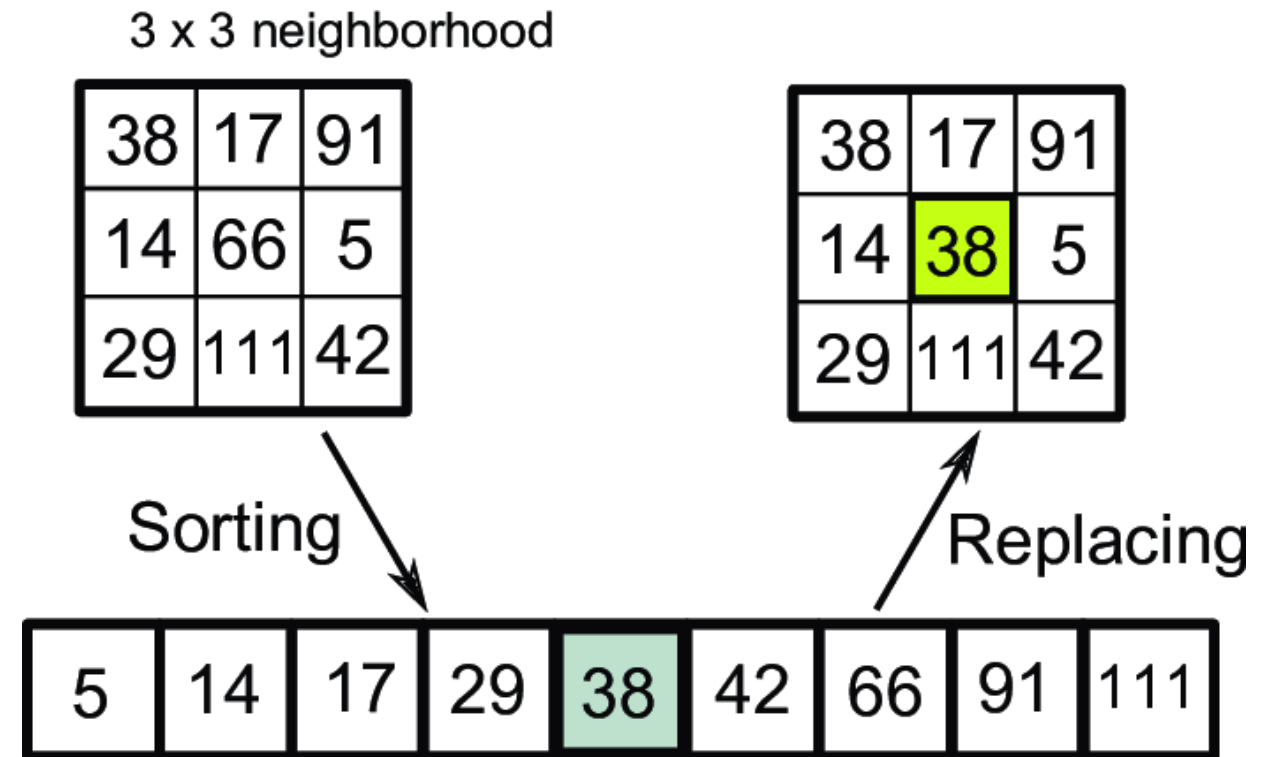
So, the new pixel value at position (2,2) is 70.



7.4.1 Order-Statistics Filters (Median Filter)

Order-statistics filters are nonlinear spatial filters whose response is based on ordering (ranking) the pixels contained in the image area encompassed by the filter, and then replacing the value of the center pixel with the value determined by the ranking result.

The best-known example in this category is the median filter, which, as its name implies, replaces the value of a pixel by the median of the gray levels in the neighborhood of that pixel (the original value of the pixel is included in the computation of the median)



Example

Input Image:

We use a 5×5 grayscale image:

$$\text{Image} = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 60 & 70 & 255 & 90 & 100 \\ 110 & 120 & 130 & 140 & 150 \\ 160 & 255 & 180 & 190 & 200 \\ 210 & 220 & 230 & 240 & 250 \end{bmatrix}$$

Central Pixel: Position (2,3)

The 3×3 window centered at pixel (2,3) is:

$$\begin{bmatrix} 70 & 255 & 90 \\ 120 & 130 & 140 \\ 255 & 180 & 190 \end{bmatrix}$$

- ✓ Extract the pixel values in the window: [70,255,90,120,130,140,255,180,190]
- ✓ Sort the values: [70,90,120,130,140,180,190,255,255]
- ✓ Find the median (middle value):

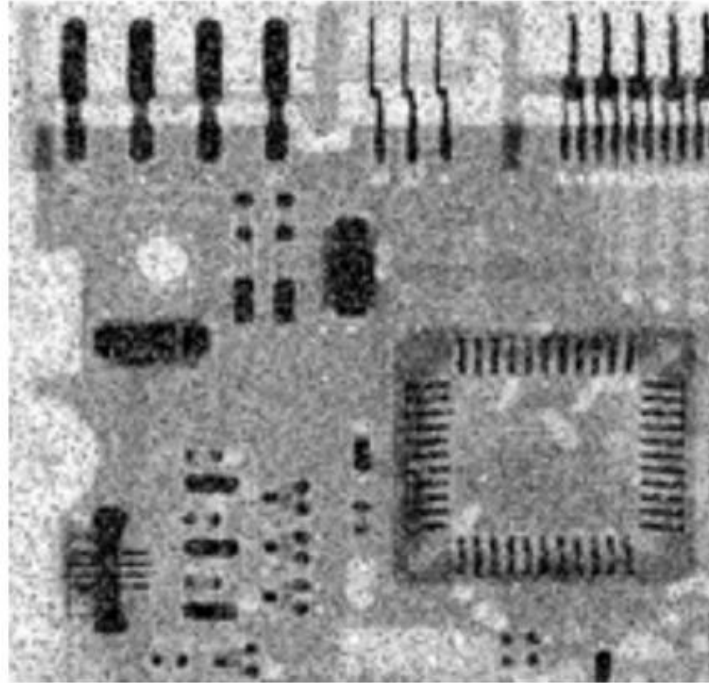
Median=140

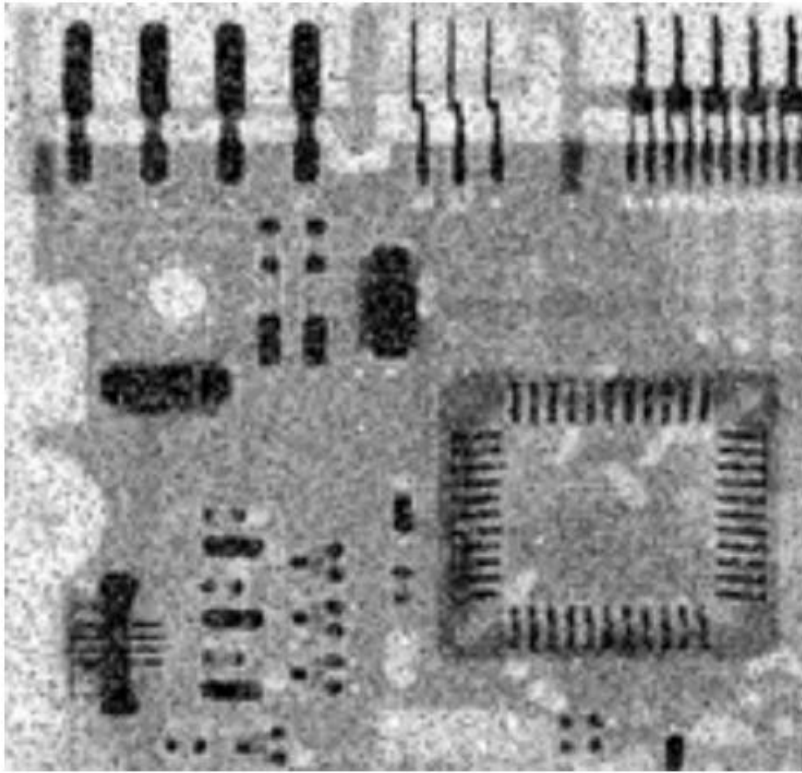
- ✓ The new pixel value at position (2,3) is 140.

$$\text{Filtered Image} = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 60 & 70 & 90 & 90 & 100 \\ 110 & 120 & 130 & 140 & 150 \\ 160 & 180 & 180 & 190 & 200 \\ 210 & 220 & 230 & 240 & 250 \end{bmatrix}$$

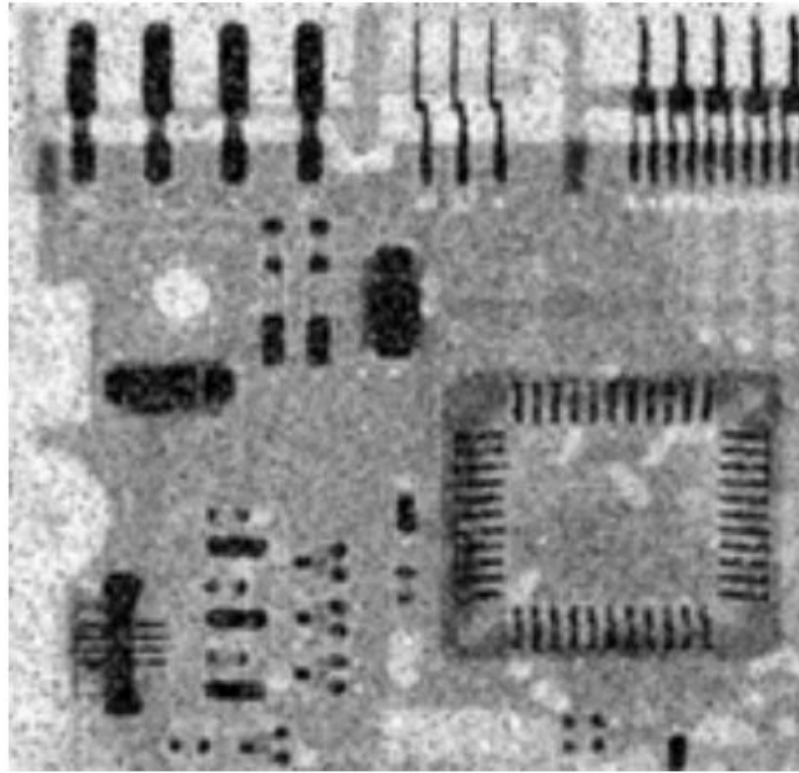
Notice that the *extreme pixel value 255 (caused by noise) is removed* and replaced by a more representative value from its neighborhood.

Median filters are particularly effective in the presence of impulse noise, also called salt-and-pepper noise because of its appearance as white and black dots superimposed on an image.

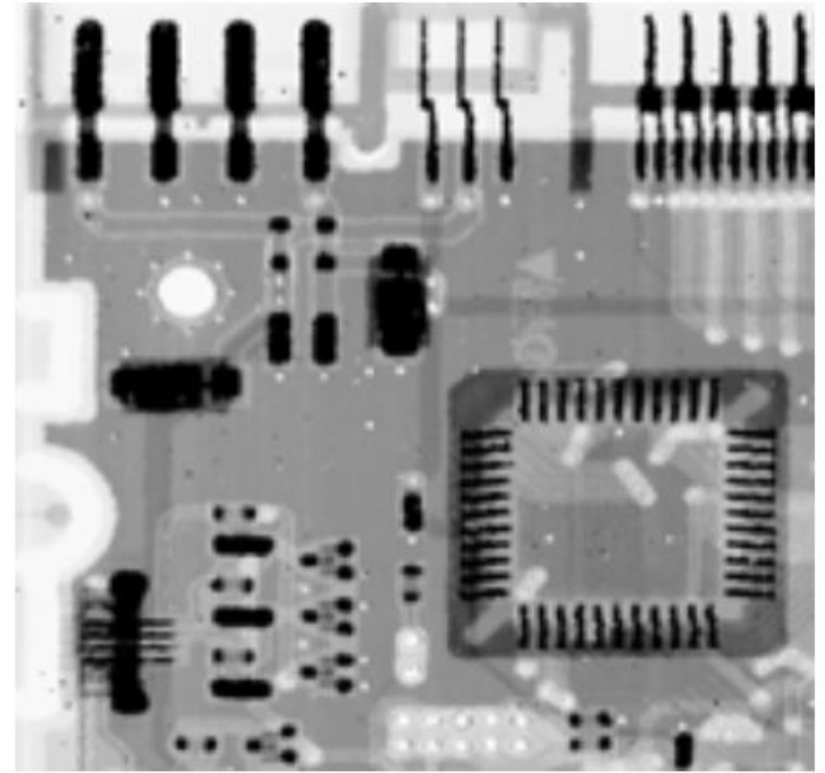




(a) X-ray image of circuit board corrupted by salt-and-pepper noise



(b) Noise reduction with a 3 X 3 averaging mask.



(c) Noise reduction with a 3 X 3 median filter.

7.4.2 Min Filters

It replaces each pixel's value with the minimum value from its neighborhood defined by a specified window (e.g., 3×3). It is primarily used for reducing noise, specifically for removing bright spots or reducing highlights (e.g., salt noise in salt-and-pepper noise).

Neighborhood Values :

35, 47, 89, 99, 103, 125, 135, 162, 198

Minimum Value:

35

135	125	198	148	53
47	35	162	100	125
89	99	103	118	200
107	234	169	59	255
184	124	211	167	236

Input Image



35	35	53
35	35	59
89	59	59

Neighborhood Window

Example

Input Image:

We use a 5×5 grayscale image:

$$\text{Image} = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 60 & 70 & 255 & 90 & 100 \\ 110 & 120 & 130 & 140 & 150 \\ 160 & 255 & 180 & 190 & 200 \\ 210 & 220 & 230 & 240 & 250 \end{bmatrix}$$

Central Pixel: Position (2,3)

The 3×3 window centered at pixel (2,3) is:

$$\begin{bmatrix} 70 & 255 & 90 \\ 120 & 130 & 140 \\ 255 & 180 & 190 \end{bmatrix}$$

1. Extract the pixel values in the window:

[70, 255, 90, 120, 130, 140, 255, 180, 190]

2. Find the **minimum value**:

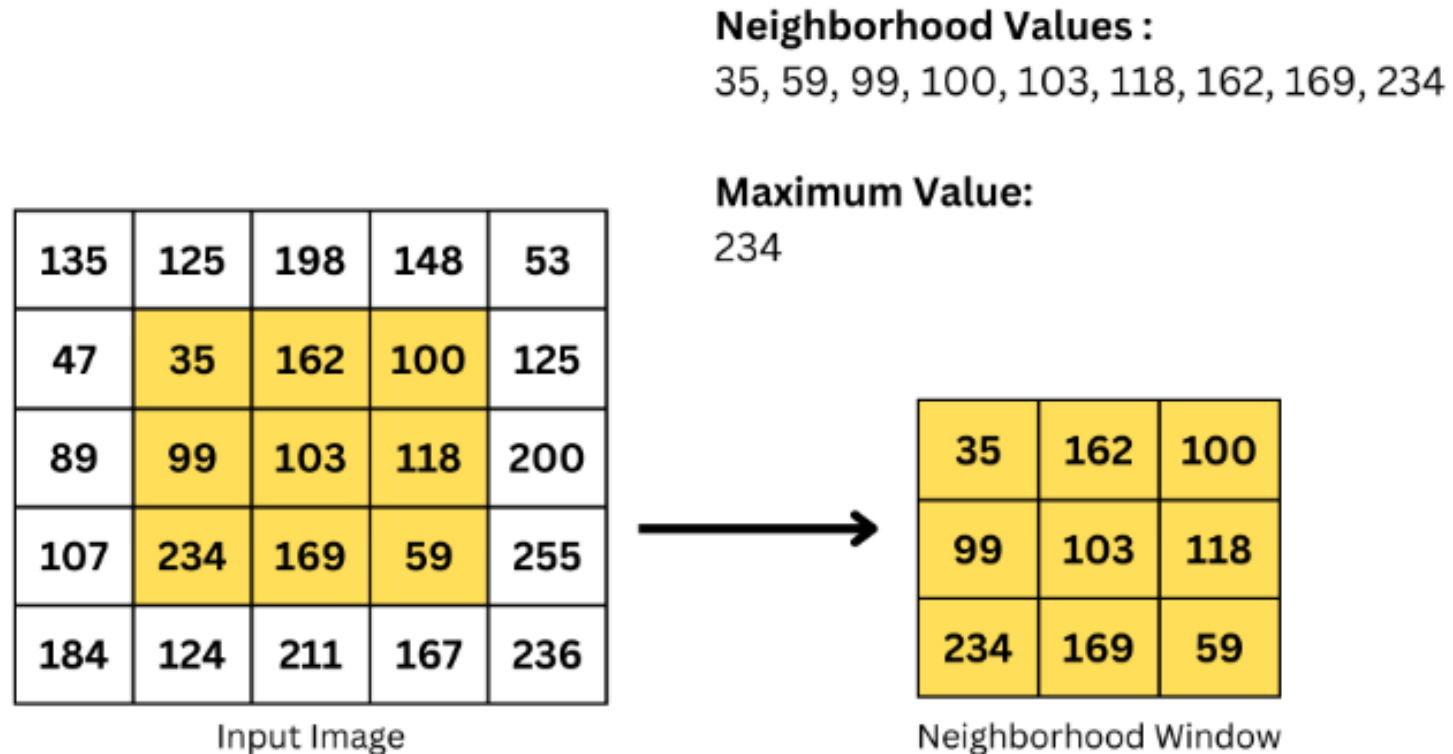
Minimum = 70

Filtered Image =
$$\begin{bmatrix} 10 & 10 & 20 & 30 & 40 \\ 10 & 10 & 20 & 30 & 40 \\ 60 & 60 & 70 & 90 & 90 \\ 110 & 110 & 120 & 130 & 140 \\ 160 & 160 & 180 & 190 & 200 \end{bmatrix}$$



7.4.3 Max Filters

The Max Filter is a non-linear filtering technique that replaces each pixel value in an image with the maximum value from its neighborhood, defined by a specific window size (e.g., 3×3 , 5×5). It is commonly used to reduce dark noise (e.g., pepper noise) and enhance bright regions in the image.



Example

Input Image:

We use a 5×5 grayscale image:

$$\text{Image} = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 60 & 70 & 5 & 90 & 100 \\ 110 & 120 & 130 & 140 & 150 \\ 160 & 5 & 180 & 190 & 200 \\ 210 & 220 & 230 & 240 & 250 \end{bmatrix}$$

Central Pixel: Position (2,3)

The 3×3 window centered at pixel (2,3) is:

$$\begin{bmatrix} 70 & 5 & 90 \\ 120 & 130 & 140 \\ 5 & 180 & 190 \end{bmatrix}$$

1. Extract the pixel values in the window:

$$[70, 5, 90, 120, 130, 140, 5, 180, 190]$$

2. Find the **maximum value**:

$$\text{Maximum} = 190$$

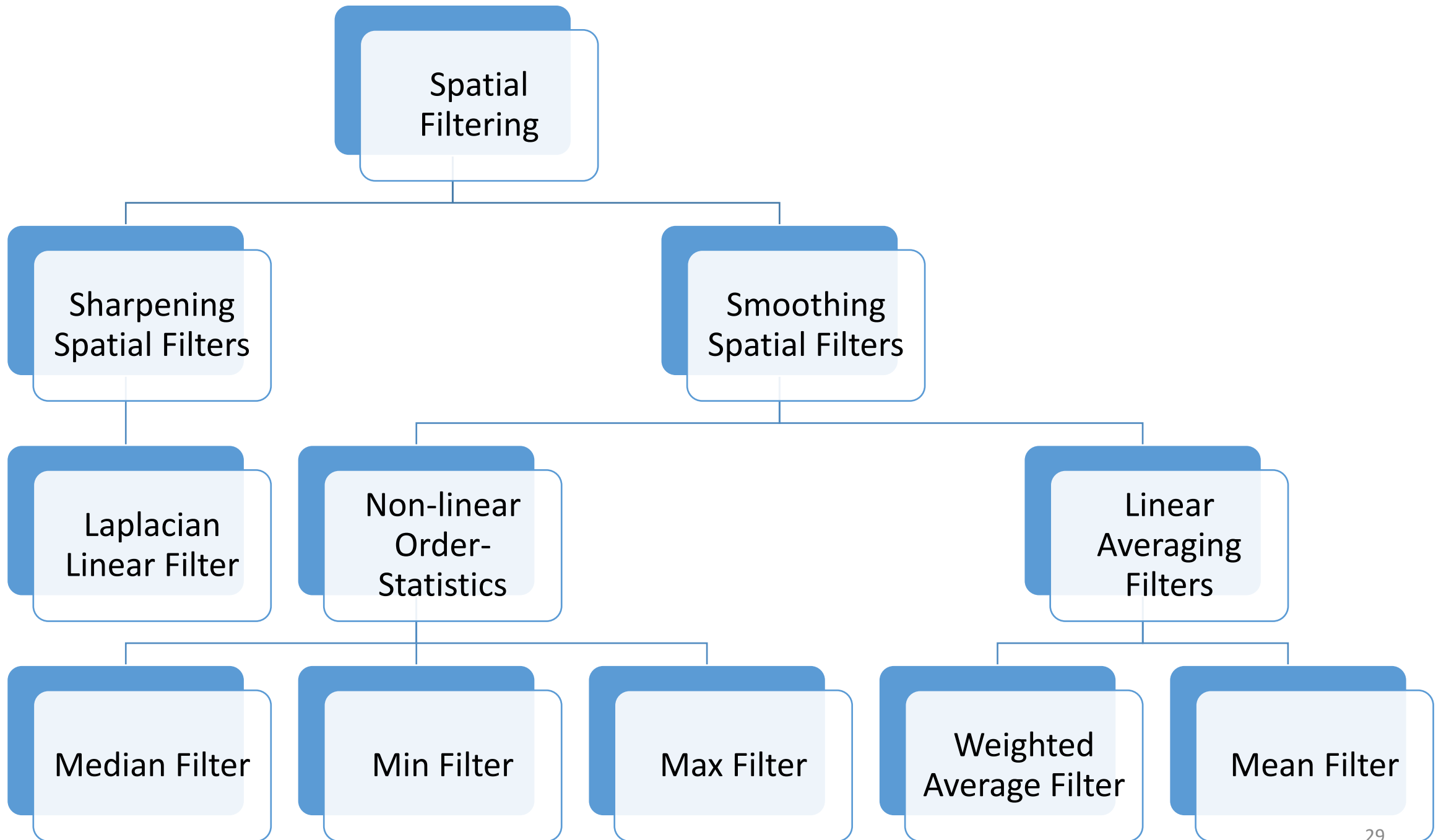
$$\text{Filtered Image} = \begin{bmatrix} 20 & 30 & 40 & 50 & 50 \\ 70 & 90 & 140 & 140 & 150 \\ 120 & 130 & 180 & 190 & 200 \\ 160 & 180 & 230 & 240 & 250 \\ 220 & 230 & 240 & 250 & 250 \end{bmatrix}$$



Before Max Filter



After Max Filter



7.5 Sharpening Spatial Filters

The principal objective of sharpening is to highlight fine detail in an image or to enhance detail that has been blurred, either in error or as a natural effect of a particular method of image acquisition.

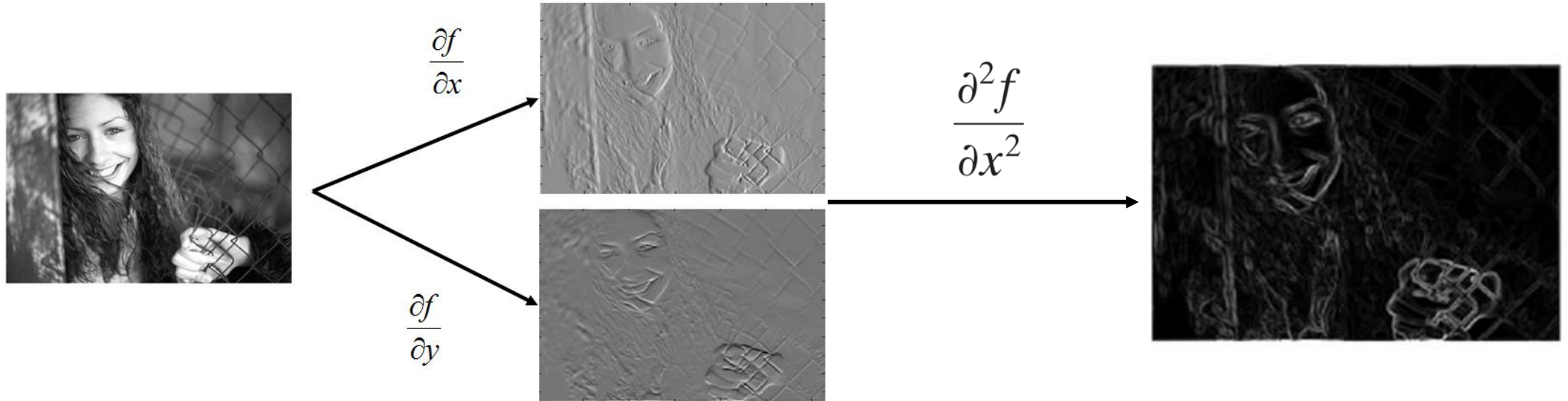
In the last section, we saw that image blurring could be accomplished in the spatial domain by pixel averaging in a neighborhood.



Since averaging is analogous to integration, it is logical to conclude that sharpening could be accomplished by spatial differentiation.

Fundamentally, the strength of the response of a derivative operator is proportional to the degree of discontinuity of the image at the point at which the operator is applied.

Thus, image differentiation enhances edges and other discontinuities (such as noise) and deemphasizes areas with slowly varying gray-level values.



The derivatives of a digital function are defined in terms of differences. There are various ways to define these differences.

However, we require that any definition we use for a first derivative

- (1) must be zero in flat segments (areas of constant gray-level values);
- (2) must be nonzero at the onset of a gray-level step or ramp; and
- (3) must be nonzero along ramps.

Similarly, any definition of a second derivative

- (1) must be zero in flat areas;
- (2) must be nonzero at the onset and end of a gray-level step or ramp; and
- (3) must be zero along ramps of constant slope.

A basic definition of the first-order derivative of a one-dimensional function $f(x)$ is the difference

$$\frac{\partial f}{\partial x} = f(x + 1) - f(x)$$

$f(x)$: Represents the value of the function (or intensity of a pixel) at position x .

$f(x+1)$: Represents the value of the function at the next position.

Difference ($f(x+1)-f(x)$): Measures the change in $f(x)$ between two adjacent points.

It detects edges by highlighting regions with large changes in intensity.

Example:

Suppose: $f(x)=[10,15,20,18,12]$

Compute $f'(x)$:

Using the formula $f'(x) = f(x+1) - f(x)$:

1. At $x = 1$: $f'(1) = f(2) - f(1) = 15 - 10 = 5$
2. At $x = 2$: $f'(2) = f(3) - f(2) = 20 - 15 = 5$
3. At $x = 3$: $f'(3) = f(4) - f(3) = 18 - 20 = -2$
4. At $x = 4$: $f'(4) = f(5) - f(4) = 12 - 18 = -6$

Result:

$$f'(x) = [5, 5, -2, -6]$$

Similarly, we define a second-order derivative as the difference

$$\frac{\partial f}{\partial x} = f(x + 1) - f(x)$$

$$\frac{\partial^2 f}{\partial x^2} = f[f(x + 1) - f(x)] - [f(x) - f(x - 1)]$$

$$\frac{\partial^2 f}{\partial x^2} = f(x + 1) + f(x - 1) - 2f(x)$$

It highlights regions of rapid intensity change and is often used to detect edges, corners, or finer details in signals or images.

Example

Input Data:

Let:

$$f(x) = [10, 20, 15, 25, 30]$$

Compute $f''(x)$:

Using the formula $f''(x) = f(x+1) - 2f(x) + f(x-1)$:

1. At $x = 2$:

$$f''(2) = f(3) - 2f(2) + f(1) = 15 - 2(20) + 10 = -15$$

2. At $x = 3$:

$$f''(3) = f(4) - 2f(3) + f(2) = 25 - 2(15) + 20 = 15$$

3. At $x = 4$:

$$f''(4) = f(5) - 2f(4) + f(3) = 30 - 2(25) + 15 = -5$$

Result:

$$f''(x) = [-15, 15, -5]$$

Figure (a) shows a simple image that contains various solid objects, a line, and a single noise point.

Figure (b) shows a horizontal gray-level profile (scan line) of the image along the center and including the noise point.

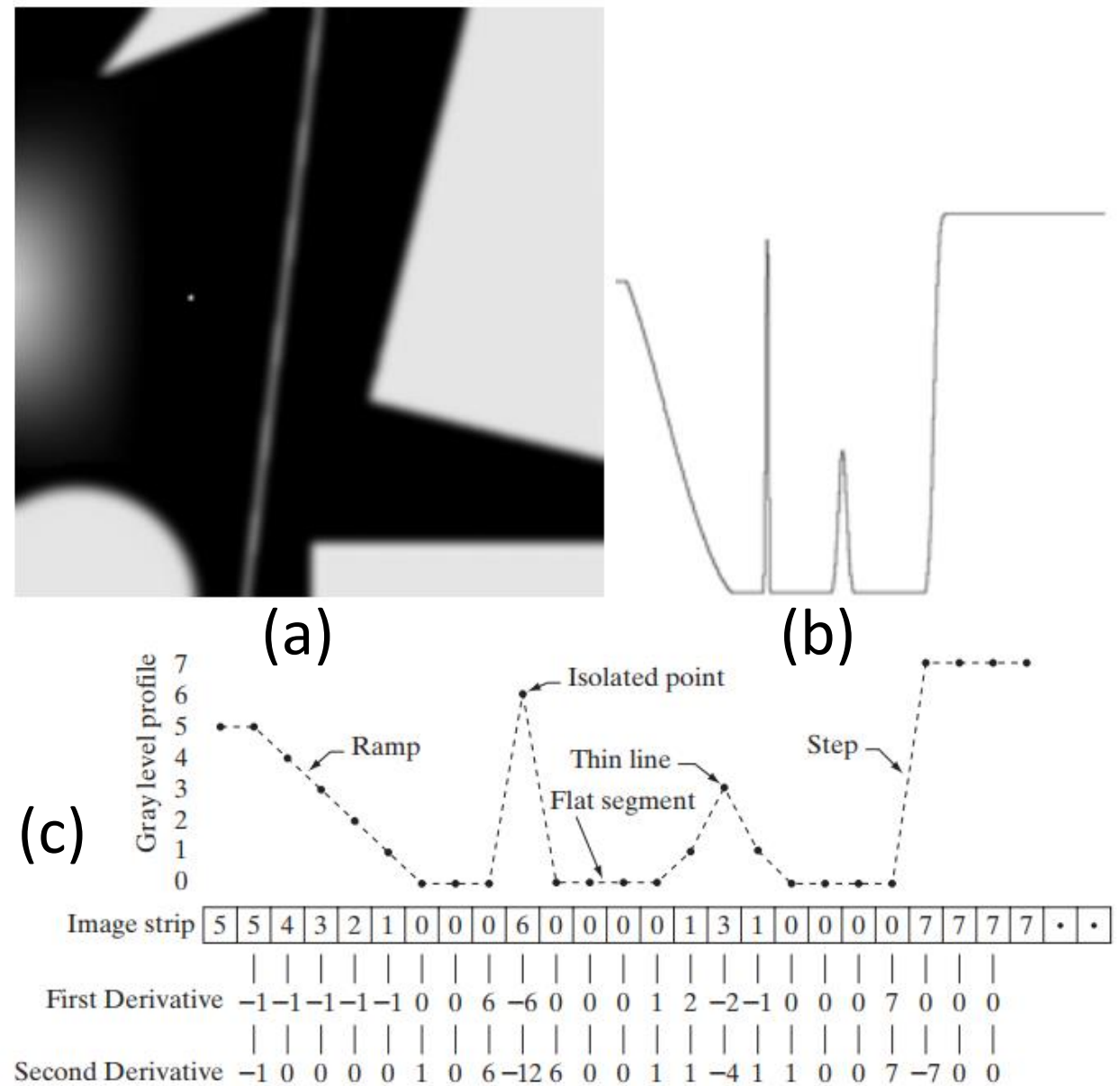
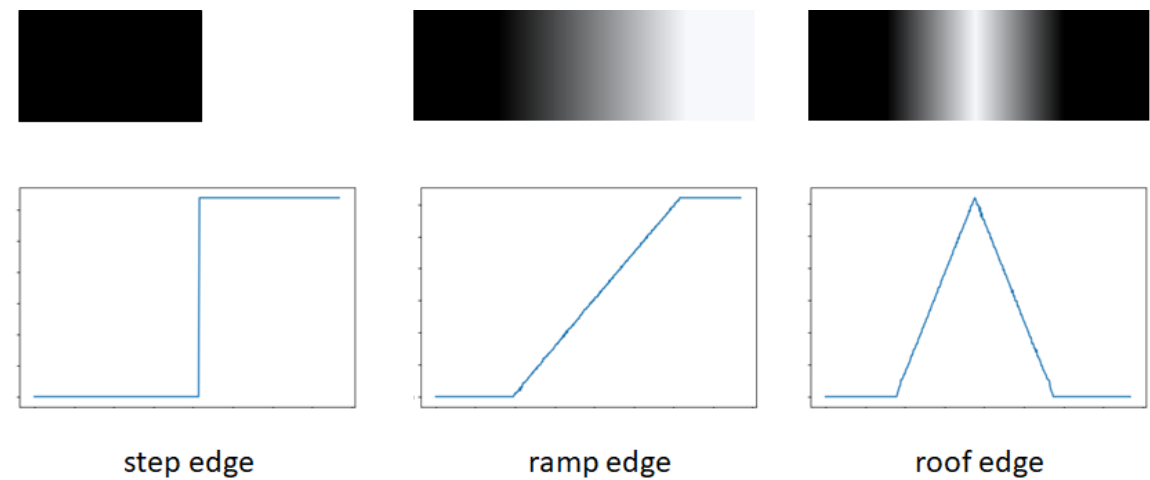
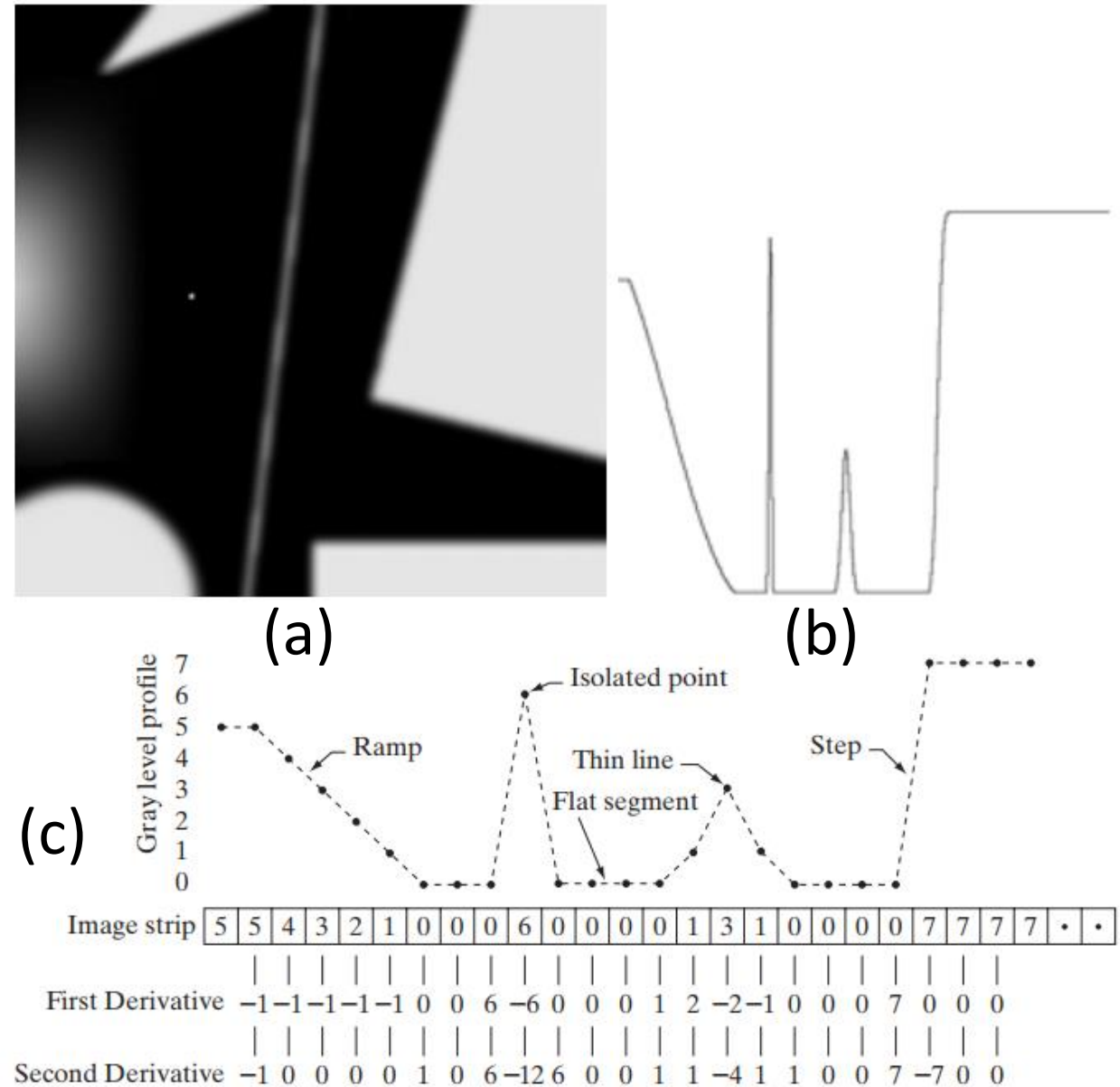


Figure (c) shows a simplification of the profile, with just enough numbers to make it possible for us to analyze how the first- and second-order derivatives behave as they encounter a noise point, a line, and then the edge of an object.

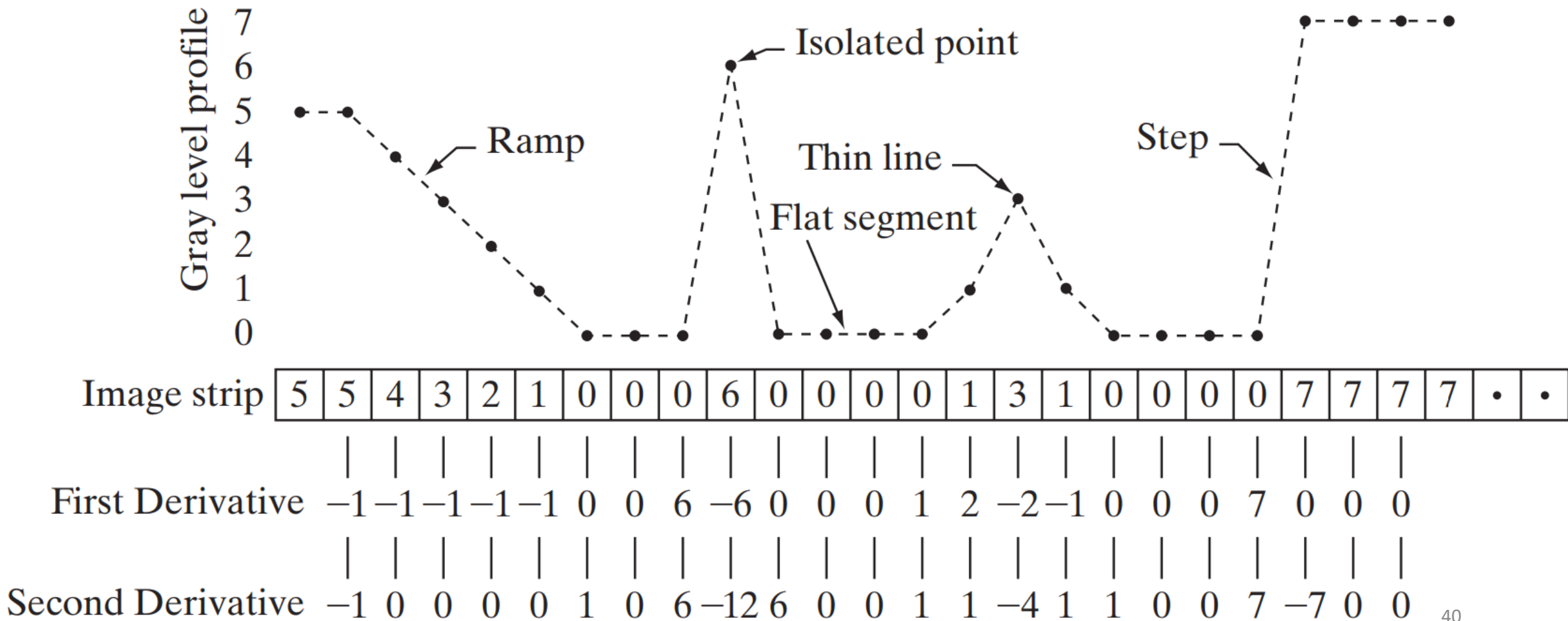
$$\frac{\partial f}{\partial x} = f(x + 1) - f(x)$$

$$\frac{\partial^2 f}{\partial x^2} = f[f(x+1) - f(x)] - [f(x) - f(x-1)]$$



$$\frac{\partial f}{\partial x} = f(x + 1) - f(x)$$

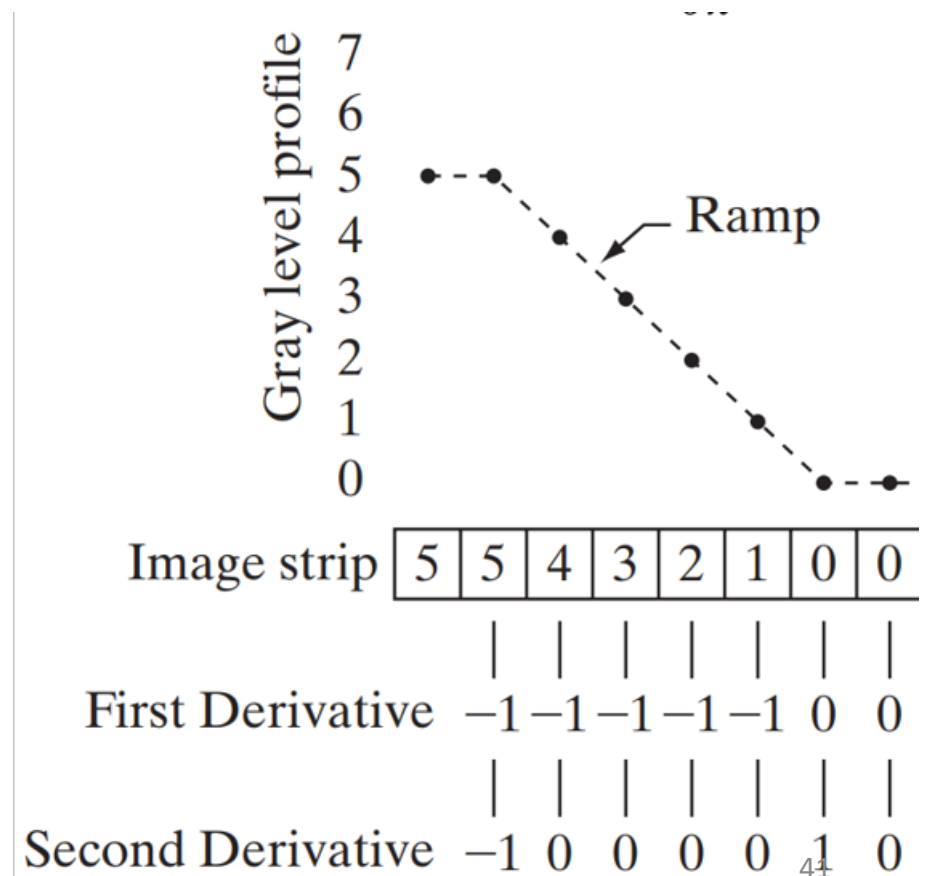
$$\frac{\partial^2 f}{\partial x^2} = f[f(x + 1) - f(x)] - [f(x) - f(x - 1)]$$



Let us consider the properties of the first and second derivatives as we traverse the profile from left to right.

- ❑ First, we note that the first-order derivative is nonzero along the entire ramp, while the second-order derivative is nonzero only at the onset and end of the ramp.

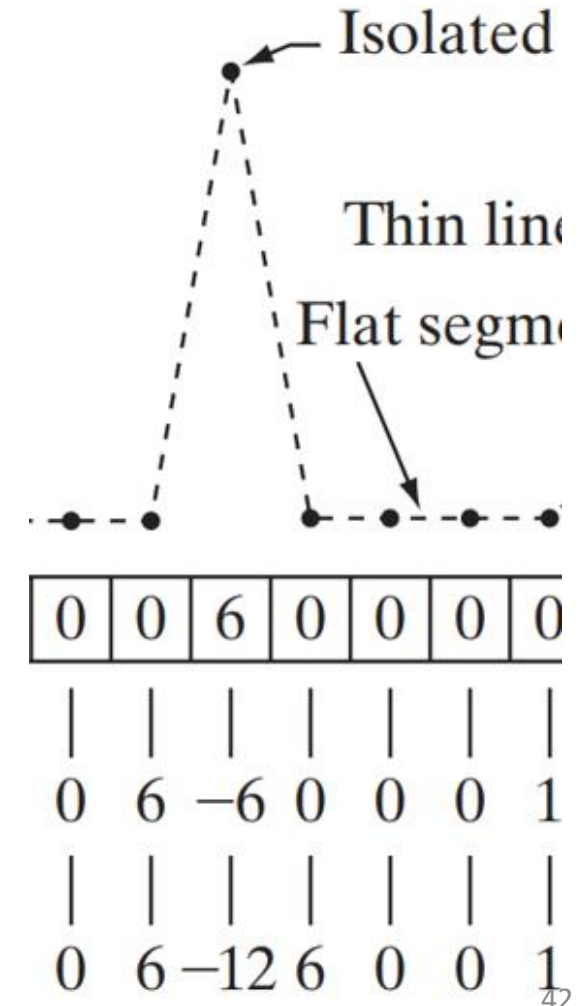
Because edges in an image resemble this type of transition, we conclude that first-order derivatives produce “thick” edges and second-order derivatives, much finer ones.



❑ Next we encounter the isolated noise point. Here, the response at and around the point is *much stronger for the second-* than for the first-order derivative. Of course, this is not unexpected.

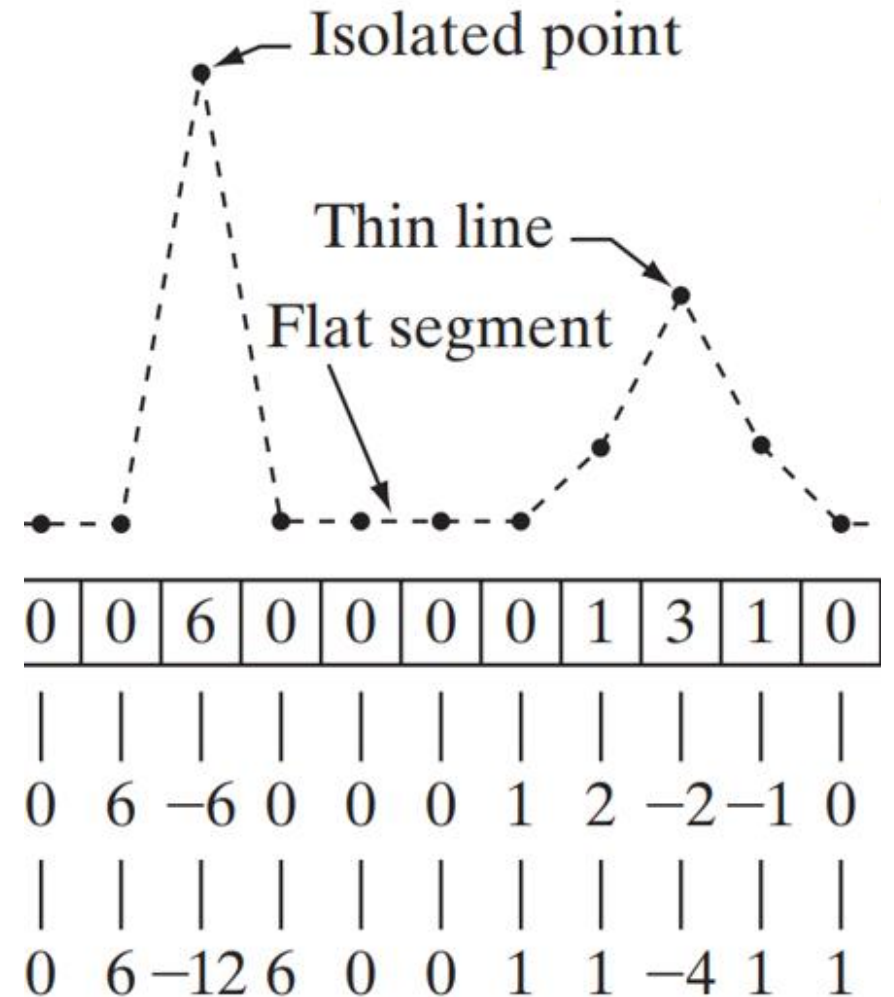
A second-order derivative is much more aggressive than a first-order derivative *in enhancing sharp changes*.

Thus, we can expect a second-order derivative to enhance fine detail (including noise) much more than a first-order derivative.



❑ The thin line is a fine detail, and we see essentially the same difference between the two derivatives.

If the maximum gray level of the line had been the same as the isolated point, the response of the second derivative would have been stronger for the latter.

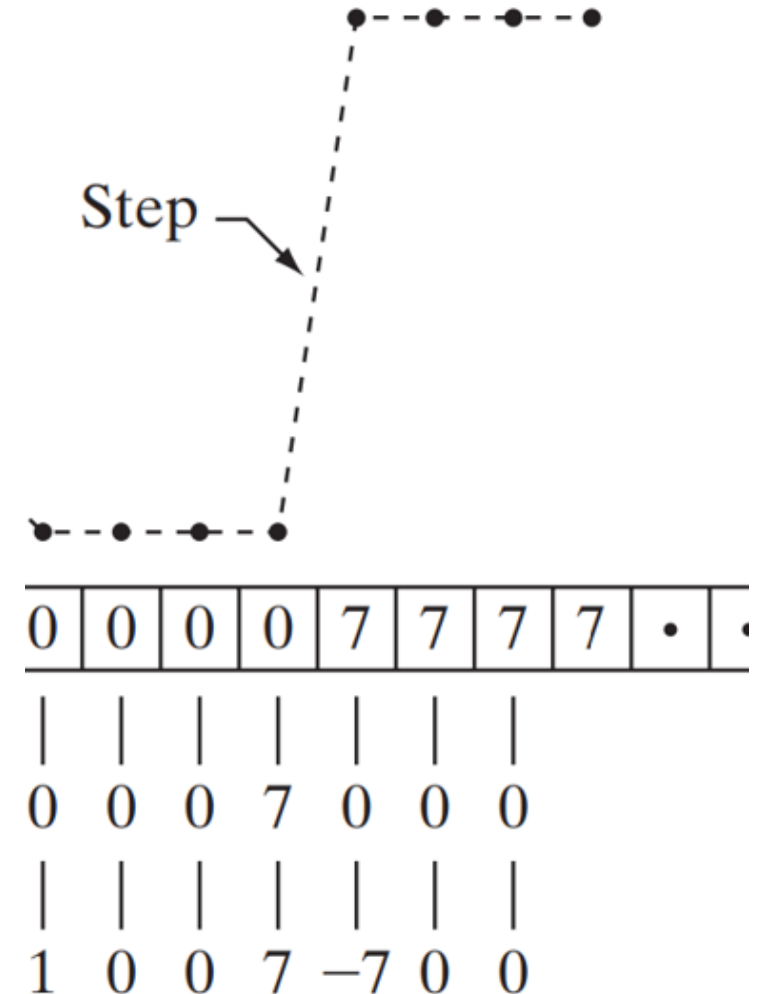


❑ Finally, in this case, the response of the two derivatives is the same at the gray-level step (in most cases when the transition into a step is not from zero, the second derivative will be weaker).

We also note that the second derivative has a transition from positive back to negative.

In an image, this shows as a thin double line.

This “double-edge” effect is an issue that will be important in Chapter 10, where we use derivatives for edge detection.



In summary, comparing the response between first- and second-order derivatives, we arrive at the following conclusions.

- (1) First-order derivatives generally produce thicker edges in an image.
 - (2) Second-order derivatives have a stronger response to fine detail, such as thin lines and isolated points.
 - (3) First order derivatives generally have a stronger response to a gray-level step.
 - (4) Second-order derivatives produce a double response at step changes in gray level.
- In most applications, the second derivative is better suited than the first derivative for image enhancement because of the ability of the former to enhance fine detail.